Skill 10

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

void builtin_pwd()
{
   char cwd[PATH_MAX];

   if (getcwd(cwd, sizeof(cwd)) != NULL)
      printf("Current directory: %s\n", cwd);
   else
      perror("pwd");
}

void builtin_exit()
{
   printf("Cleaning up resources...\n");
   printf("Saving state...\n");
   printf("Exiting shell...\n");

   exit(0);
}

int main()
{
```

```
    char command[100];

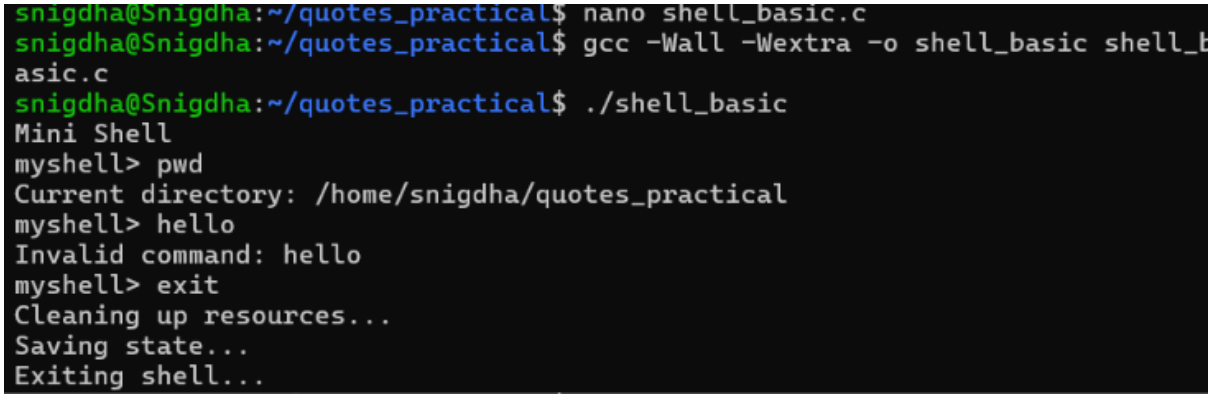
    printf("Mini Shell\n");

    while (1)
    {
        printf("myshell> ");

        if (scanf("%99s", command) != 1)
            break;

        if (strcmp(command, "pwd") == 0)
        {
            builtin_pwd();
        }
        else if (strcmp(command, "exit") == 0)
        {
            builtin_exit();
        }
        else
        {
            printf("Invalid command: %s\n", command);
        }
    }

    return 0;
}
```




```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#include <unistd.h>

#include <sys/wait.h>


int valid_name(char *name)

{

  if (name[0] == '\0')

    return 0;


  if (!isalpha(name[0]) && name[0] != '_')

    return 0;


  for (int i = 1; name[i] != '\0'; i++)

  {

    if (!isalnum(name[i]) && name[i] != '_')

      return 0;

  }


  return 1;
```

```
}

void builtin_export(char *assignment)
{
   char *equals = strchr(assignment, '=');

   if (equals == NULL)
   {
      printf("export: invalid syntax\n");
      return;
   }

   *equals = '\0';

   char *name = assignment;
   char *value = equals + 1;

   if (!valid_name(name))
   {
      printf("export: invalid variable name\n");
      return;
   }

   if (setenv(name, value, 1) != 0)
   {
      perror("export");
      return;
   }
```

```
    printf("Exported: %s=%s\n", name, value);
}

void test_child()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        char *value = getenv("MYVAR");

        if (value != NULL)
            printf("Child received MYVAR=%s\n", value);
        else
            printf("Child: MYVAR not found\n");

        exit(0);
    }
    else
    {
        wait(NULL);
```

```
    }
}

int main()
{
    char input[200];

    printf("Mini Shell - Export Test\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

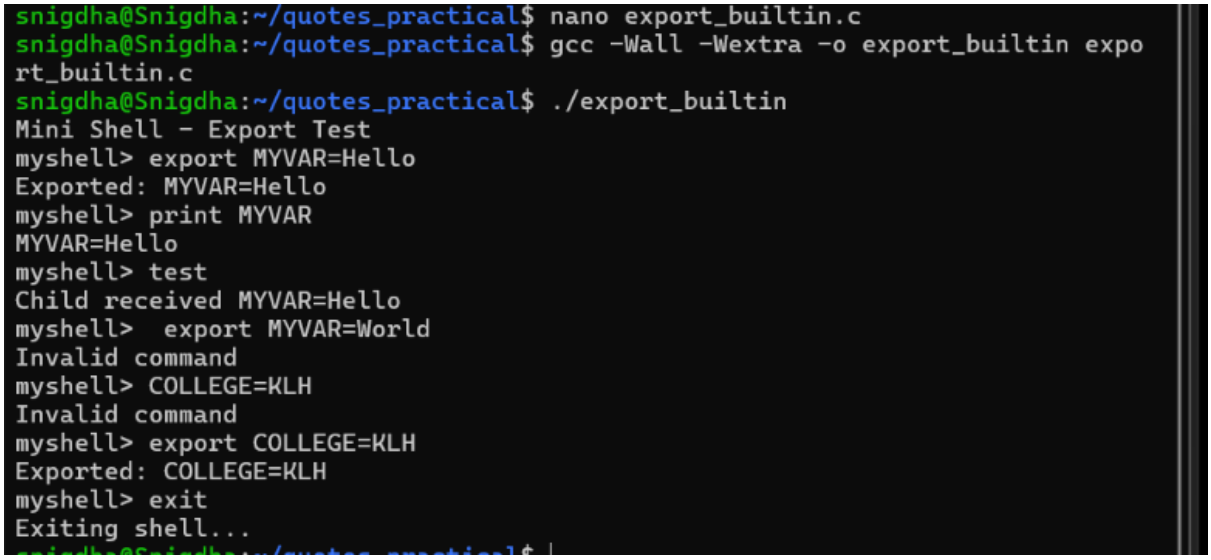
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        else if (strncmp(input, "export ", 7) == 0)
        {
            builtin_export(input + 7);
        }
        else if (strcmp(input, "test") == 0)
```

```
        {
            test_child();
        }
        else if (strncmp(input, "print ", 6) == 0)
        {
            char *value = getenv(input + 6);

            if (value != NULL)
                printf("%s=%s\n", input + 6, value);
            else
                printf("Variable not found\n");
        }
        else
        {
            printf("Invalid command\n");
        }
    }

    return 0;
}
```


snigdha@Snigdha:~/quotes_practical$ nano export_builtin.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -o export_builtin expo
rt_builtin.c
snigdha@Snigdha:~/quotes_practical$ ./export_builtin
Mini Shell - Export Test
myshell> export MYVAR=Hello
Exported: MYVAR=Hello
myshell> print MYVAR
MYVAR=Hello
myshell> test
Child received MYVAR=Hello
myshell> export MYVAR=World
Invalid command
myshell> COLLEGE=KLH
Invalid command
myshell> export COLLEGE=KLH
Exported: COLLEGE=KLH
myshell> exit
Exiting shell...